

infix $a + b$

post fix $ab +$

infix

post fix

$$a + (b * c) = (a) + (b * c)$$

abc * +

$$\underline{a + b - c * c / d * f - g}$$

↓
algorithm:

Algorithm $\rightarrow$ infix to postfix.

a) create a stack

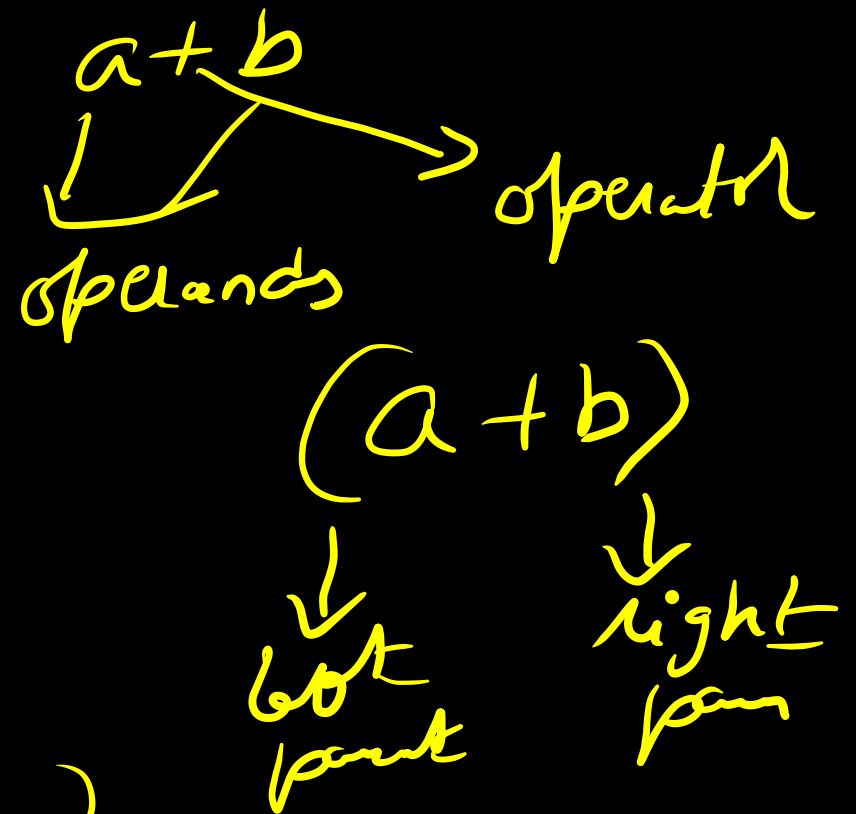
b) For each character 't' in i/p

{ if (t is an operand)
 o/p t;

else if (t is a right parenthesis)

{ pop and o/p tokens until a
 left parenthesis is popped (but not o/p)

}

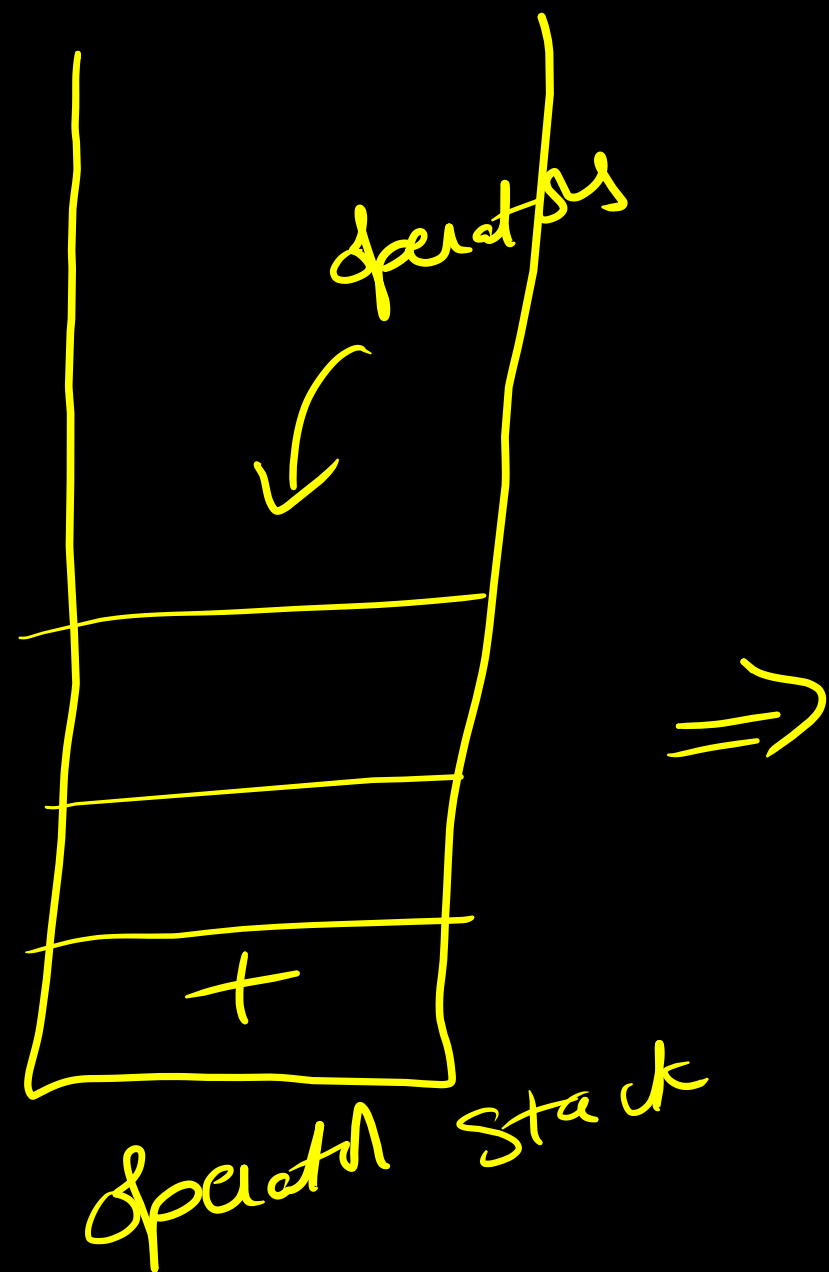


else if t is an operator or left parenthesis
pop and o/p tokens until one of the
lower priority than ' t ' is encountered
or a left parenthesis is encountered
or stack is empty

push t

}

}



$a + b * c.$
 $\uparrow \quad \uparrow$
always higher priority operator should
sit on top of low priority operator.

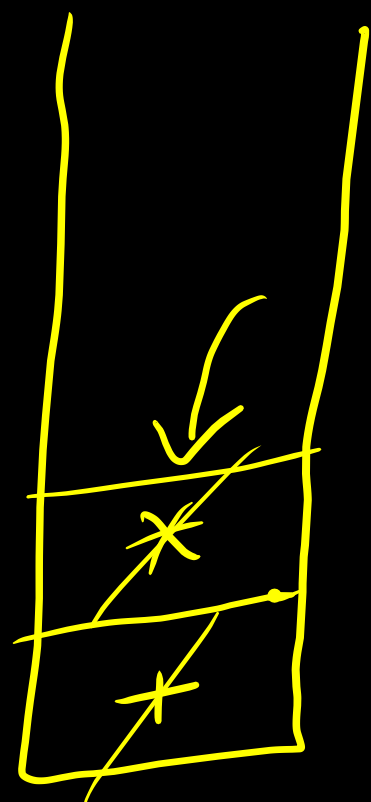
a
 $\uparrow$

$+$
 $\uparrow$

b
 $\uparrow$

$*$
 $\uparrow$

c
 $\uparrow$



$+c*$

$abc*+$

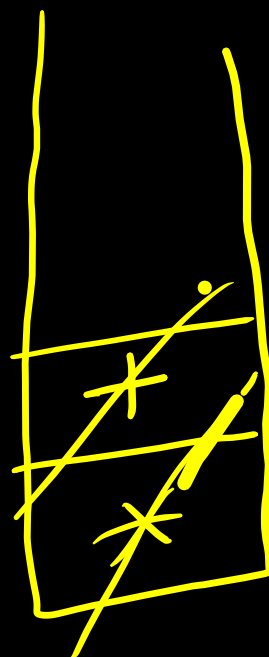
a
 $\uparrow$

$*$
 $\uparrow$

b
 $\uparrow$

$+$
 $\uparrow$

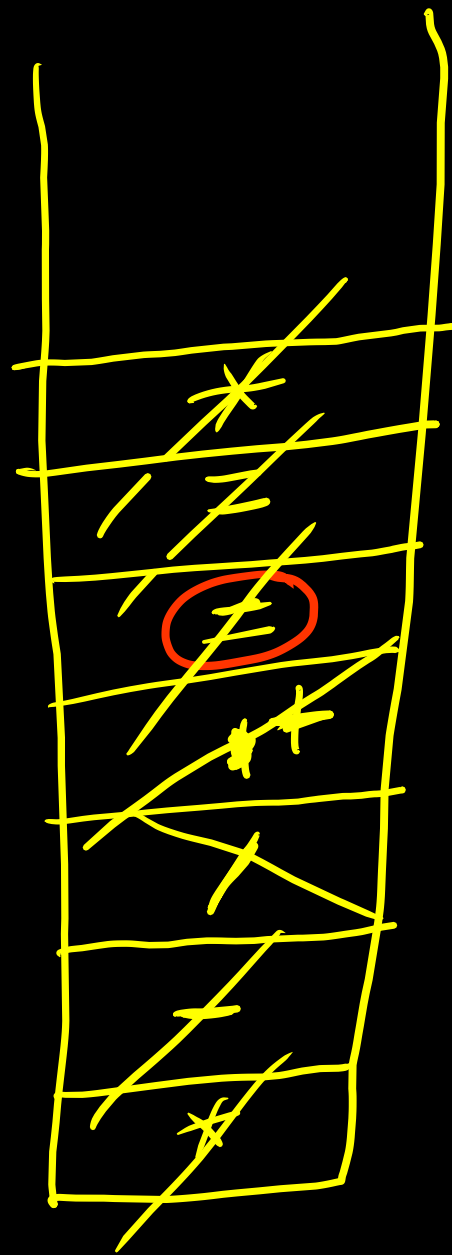
c
 $\uparrow$



$abc*+$

$(d * a - b / c + e = f \equiv g * h)$ ✓ higher priority on set on
 $\uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow$ low priority.

$d a * b c / - e + f g h * ==$



$= < =$
 $- > +$
 $= =$
 $+ +$
 $- +$

Same priority
 associating

$$\left(\begin{array}{c} t \\ \hline \hline \end{array} a \right) \begin{array}{c} t \\ \hline \hline \end{array}$$

t in left assoc

Infix $\rightarrow$ post fix $\rightarrow$ evaluate post fix.

evaluate Infix $\rightarrow$ multiple scans are required

$a + b - c * d / e * f$

I $\rightarrow$

II $\rightarrow$

III $\rightarrow$

Infix - postfix $\rightarrow$ 1 scan & 1 pass

postfix evaluation $\rightarrow$ 1 pass.

$a + b = c \& d \& e = f = g = h \neq i$
 $\uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \downarrow$
 $a \ b \ + \ c \ d \ \& \ e \ \& \ f \ g \ h \ g \ \neq \ = \ = \ = \ =$

1	=
7	
7	
7	
7	
88	
8	
7	
7	

+ > =

$10/5 - 3 * 7 + 8 / 1 * 9 + 3 - 4 * 7$
 $\uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow \uparrow$

$10 \ 5 / 3 \ 7 * - 8 / 9 * + 3 + 4 \ 7 *$

*
*
*
x
+
*
+
x

5 min best

Operators	Associativity
() [] -> .	left to right
! ~ ++ -- + - * (type) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
	left to right
&&	left to right
	left to right
? :	right to left
= += -= *= /= %= &= ^= = <<= >>=	right to left
,	left to right

op < (

(< op

* > +

* > *

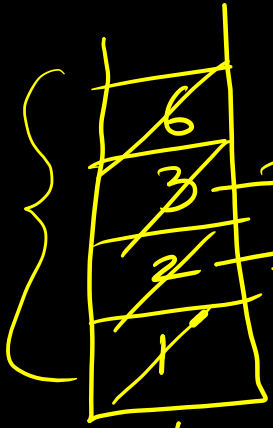
+ > *

decreasing

= < =

Infix $\rightarrow$ postfix $\rightarrow$ Evaluate a Postfix.

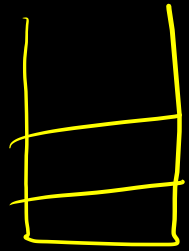
1 2 3 $\odot$ $\oplus$



$$\begin{aligned} & \xrightarrow{\text{op2}} 2 \times 3 = 6. \\ & \xrightarrow{\text{op1}} 1 + 6 = \odot 7 \checkmark \end{aligned}$$

Append Stack

Infix - postfix



operational stack.

Algorithm for postfix evaluation:

- 1) Scan the postfix from left to right
- 2) Initialize an empty stack
- 3) Repeat steps 4 and 5 till all the characters are scanned
- 4) If the scanned character is an operand, push it on to the stack
- 5)

5) If the scanned character is an operator and if the operator is unary, then pop an element from the stack. If the operator is binary, then pop two elements from the stack. After popping the elements, apply the operator to those popped elements. Push the result on to the stack.

simp dgo:

if (dp[and])

push

else if (dp[or])

{

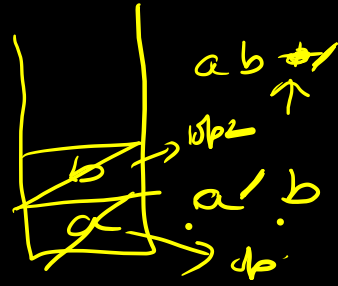
dp2 = pip(2)

dp1 = pip(1)

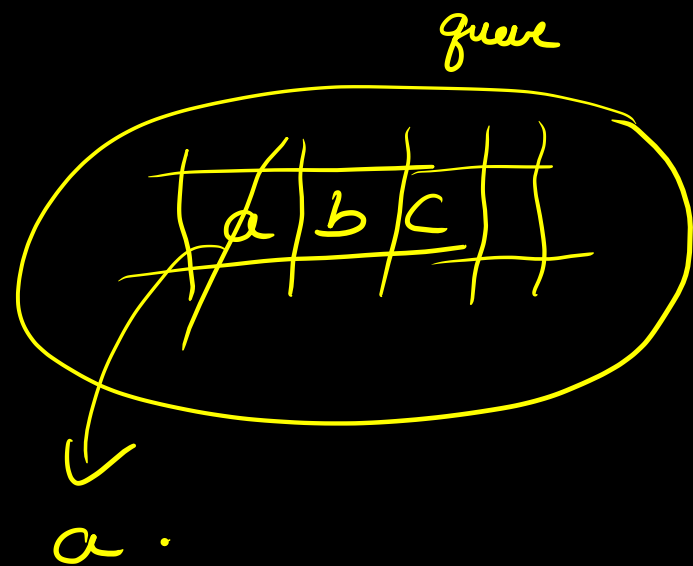
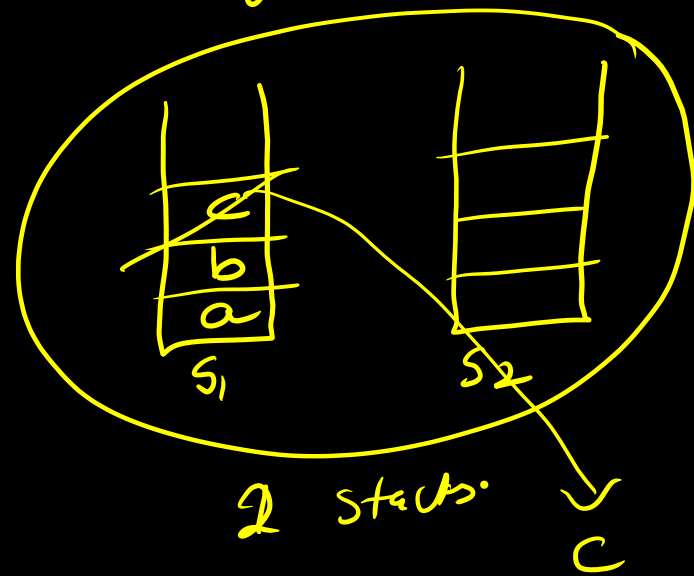
result = dp1 or dp2

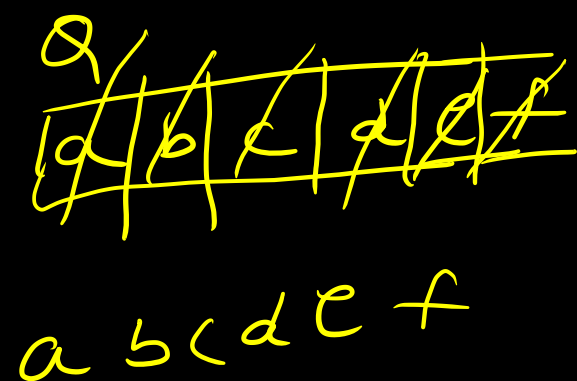
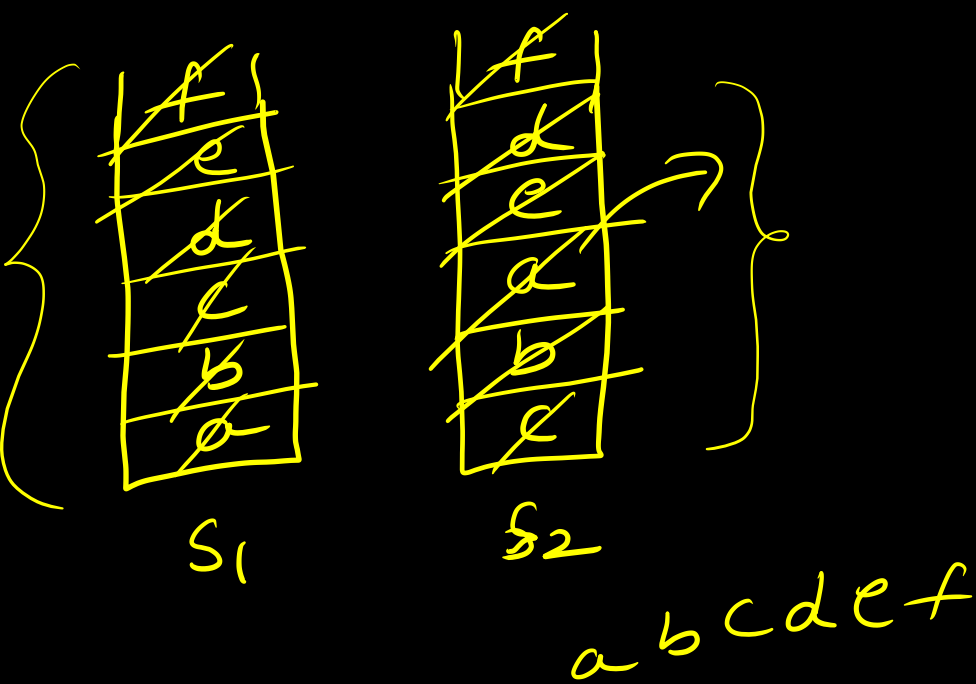
push(r)

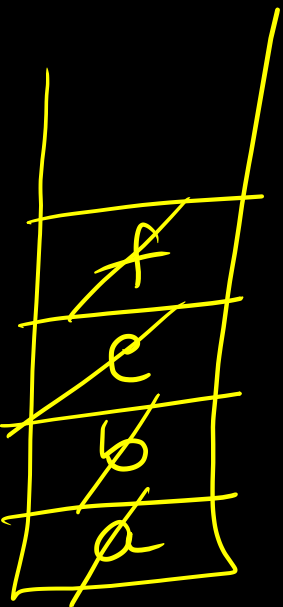
}



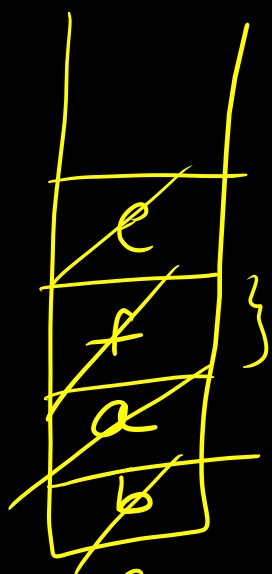
implementing queue using two stacks.







S_1
=



S_2

abef

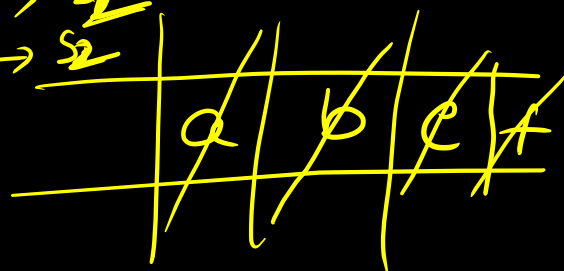
push $\rightarrow S_1$

pop $\rightarrow S_1$

push $\rightarrow S_2$

pop $\rightarrow S_2$

$$(n \neq 4) = \underline{\underline{O(n)}}$$



abef

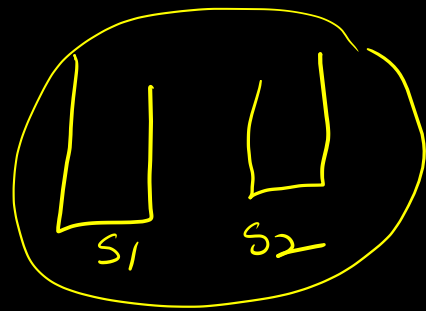
Alg:

void imax(Q, x)

{ push(s1, x)

}

~~let~~



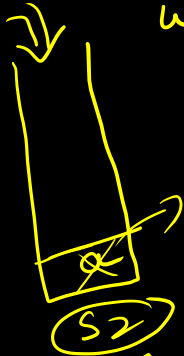
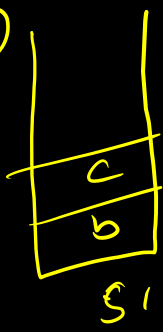
```
void delete ( Q, x)
```

```
{  
  if (! stack - empty (S2))  
    x = pop S2
```

```
  if ( stack - empty (S2)) then  
    if ( stack - empty (S1)) then  
      pf (Q is empty)  
    }
```

delete

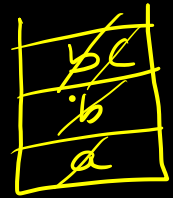
①



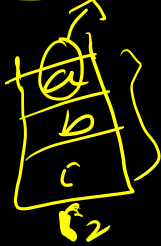
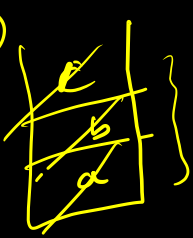
else // S_1 is not empty S_2 is empty
while (!stack empty (S_1))

{ $x = \text{pop}(S_1)$;
 push (S_2, x)

}
 $x = \underline{\underline{\text{pop}(S_2)}}$



②



$Q \rightarrow 2S$
 $\Downarrow$
 15 Eq
 4 dg
 25 Eq
 19 dg

$\Downarrow$
 $\text{pop} = ?$
 $\text{push} = ?$

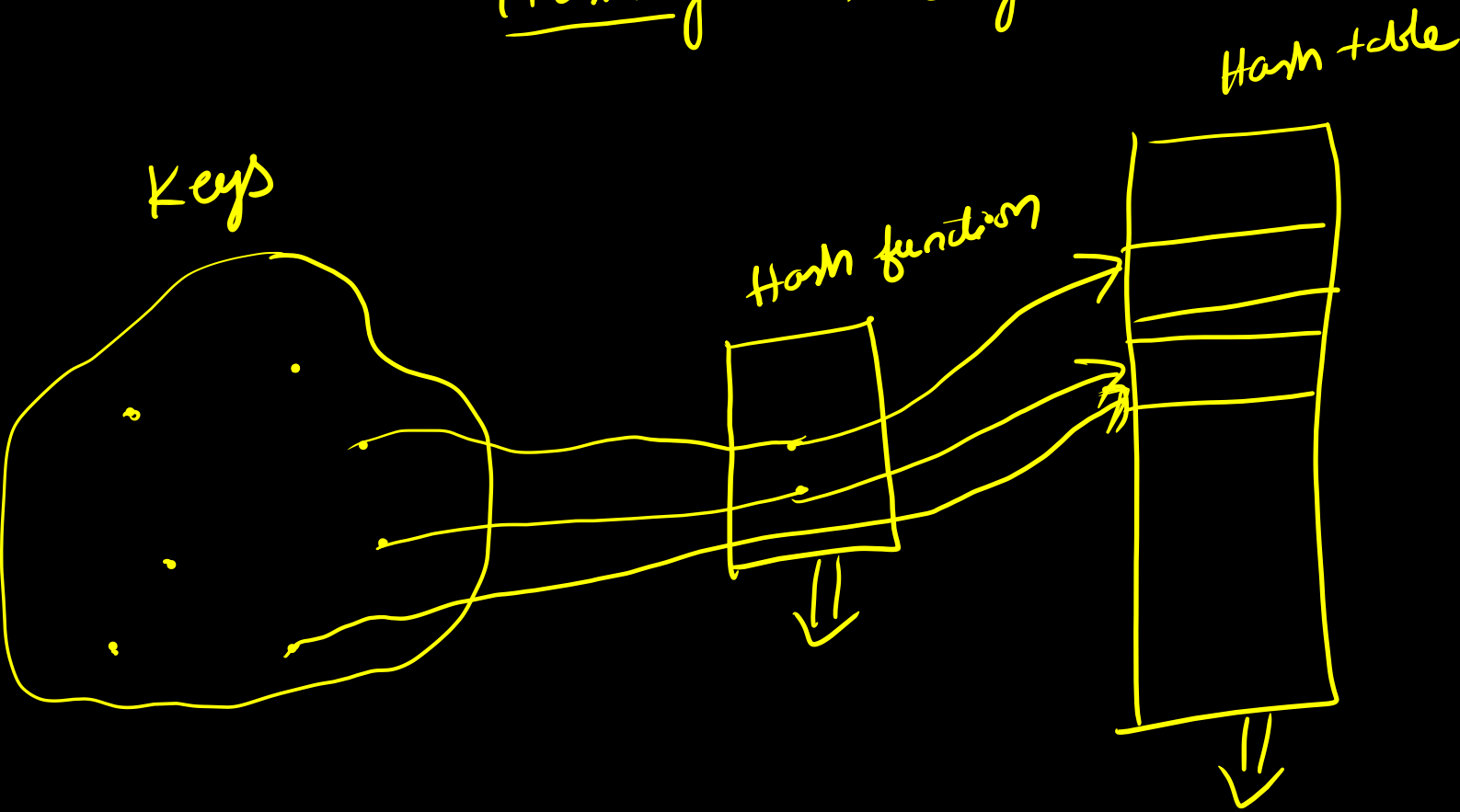


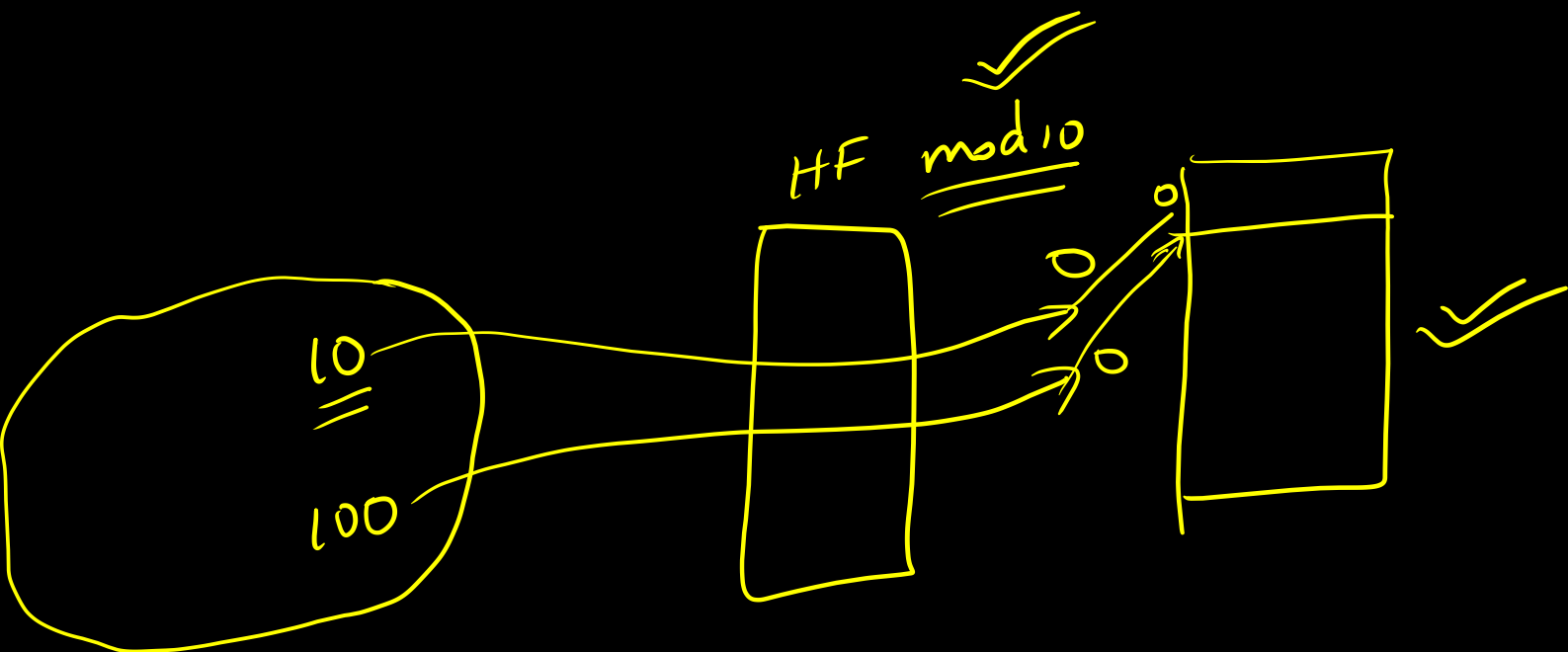
$\text{push} = 80$
 $\text{pop} = 63$

$15 \text{ equs} = 15 \text{ push}$
 1 dg $= 15 \text{ pop } (S_1)$
 $15 \text{ push } (S_2)$
 $1 \text{ pop } (S_2)$

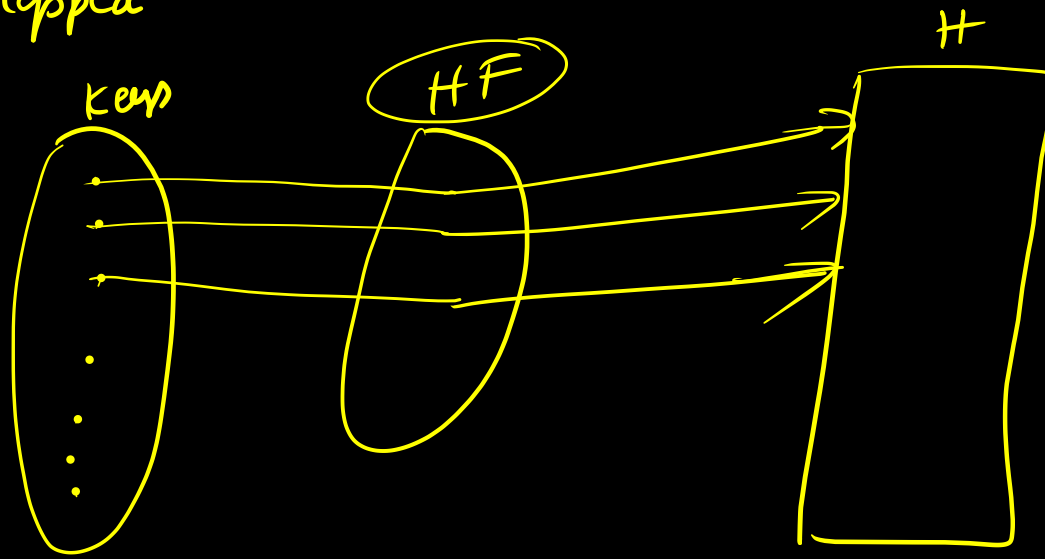
$3 \text{ dg} - 3 \text{ pop } (S_2)$
 $25 \text{ Eq} - 25 \text{ push } (S_1)$
 $11 \text{ dg} - 11 \text{ pop } (S_2)$
 $12^{\text{th}} \text{ dg} - 25 (\text{pop} - S_1)$
 $25 (\text{push} - S_2)$
 1 pop
 ~~7 dg~~ $- 7 \text{ pop}$

Hashing: $\rightarrow$ easy.





Hash function is a mapping technique where one key is mapped to one of the index in Hash table



Disadvantages of HF:

Collisions

Types of Hash functions:

- 1) Division modulo method
- 2) mid square method
- 3) Digit extraction method
- 4) Fld boundary method
- 5) Fld shifting method
- 6) Digit truncation method.

1) Division modulo method:

$$HF(\text{key}) = \text{key} \bmod \underline{\underline{m}}$$

$$m = 1000$$

$$HF(25) = 25 \bmod 1000 = 25$$

$$HF(1625) = 625$$

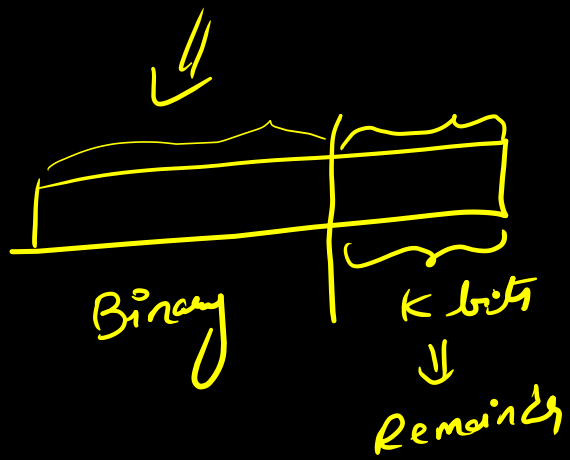
$$HF(3005) = 5$$

$$HF(4005) = 5$$

$\rightarrow$ collision

in division modulo method $\rightarrow$ don't take power of 2 as a 'm'

$$N \bmod 2^k \rightarrow \text{remainder} \Rightarrow$$



Recommended $m \rightarrow$ prime number far away from
power of '2'

~~(61)~~ $\rightarrow$
(64)

~~(511)~~
(512)

~~(701)~~ $\checkmark \rightarrow$ (512
1024)
 $\downarrow$
 2^k

0 1000000000
|-----|
keys $\rightarrow$ high

take a big number
considering HT size due

mid square method:- (uses all the digits for key Hash) →

key = 54389

↓ square

295 816 3321

↓ take mid

816
163 007

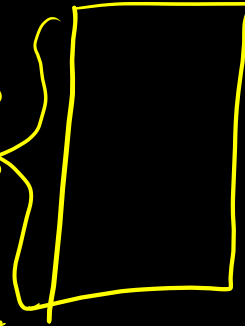
if HT has 0 - 999 values ✓

✓ 1 digit → 0-9 ⇒ 10
✓ 2 digit → 10-99 ⇒ 90

✓ 3 digit ⇒ 900

900 ✓

majority are
3 digits



Digit extraction method:-

HT [1000] (0-999)

Key = 5 8 9 4 6 7 3 2 0
 ! 5 8
 ↓ ↓
 5 6 2 .

5 6 8 9 6 12 2 1

→ collide.

Fold boundary method:-

HT[1000] [0-999]

Key = $\overset{\checkmark}{\textcircled{5\ 8\ 9}}\ 6\ 2\ 4\ \overset{\checkmark}{\textcircled{9\ 3\ 2}}$
1 2 3 4 5 6 7 8 9

$\Rightarrow \textcircled{9\ 3\ 2}\ 1\ 2\ 3\ \textcircled{5\ 8\ 9}$

589
932

1521

152
1

153 ✓

932
589

1521

153 ✓

✓ Fold shifting method: HT [1000] (0-999)

Key = 589 624 932
1 2 3 4 5 6 7 8 9

589
624
932

2145

214
5

219

Key₁ = 624 589 932

219 all digit
Best methods: mid square, Fold shifting.